

Major Exam Report

Joel Paul^{a,1}

^aRoll- No- M24DE2012
^bMail- Id- m24de2012@iitj.ac.in

IIT Jodhpur

1. Introduction

This is my Speech Understanding take home major Exam Report. I solved problems on speech synthesis, speech denoising and a speech related research idea in this report.

All my codes are implemented in python 3.11 on colab using the datasets provided. I used google drive to solve the questions and also pushed my observations, output, evaluation metrics on my drive, most of the output links in the report comes from my drive link which i made it public. Please find my google colab notebook implementation [here](#)

2. Question 1: Speech Synthesis

For this question I selected a 54-minute lecture from the Speech Understanding course, which involved significant code-switching between English and Hindi.

2.1. Deliverables

- Original Lecture Video [link](#)
- Original Lecture Audio [link](#)
- Original Transcribed Audio (trimmed to first 30 minutes to save commute and time) [LINK](#)
- Lower Resource Translated Text (TELUGU) [LINK](#)
- Cloned Output
 - GENERATED OUTPUTS CHUNKS [LINK](#)
 - FINAL GENERATED AUDIO [LINK](#)

2.2. Algorithm and Tools Used

The overall speech synthesis pipeline included the following stages:

2.2.1. Transcription

I selected a 54-minute [lecture](#) from the Speech Understanding course, which involved significant code-switching between English and Hindi. I used the [Whisper ASR model](#) for transcription due to its multilingual support. Initially i trimmed the audio file for around 30 minutes to save time and compute power. It took me around 5 and half hours to get the complete output as i was running it on Colab free CPU. Then i have observed that Whisper was able to handle code switching quite effectively. There are a few cases where the stopwords and sentences were repeated more than one. To handle these cases i tried breaking the transcription into smaller, coherent parts before translating them. Overall, the system handled code-switching effectively, especially during the transcription phase. The model produced a high-quality transcription that accurately preserved both English and Hindi segments.

2.2.2. Post Processing

I post-processed the transcript using a regular expression-based filter to remove filler words such as “um”, “uh”, and “basically” and other stopwords in hindi to achieve better clarity. The complete workflow for processing the lecture audio involved multiple steps, each designed to handle the unique challenges of code-switched input and low-resource language translation.

2.2.3. Translation

The cleaned transcript was translated into Telugu using the using several models available in hugging face but i found [Facebook](#) and [Googles translation](#) quite impressive. I used google libraries too but

there is a limit in translating text. So i have divided the data into chunks to get good translation results. To maintain translation accuracy, I segmented the text into smaller, coherent chunks before translation. This approach helped preserve context and reduce mis-translations in the target language.

2.2.4. Audio Generation

To Generate the audio using translated telugu text i tried several models again. I tried to synthesize the speech using [Coqui TTS](#) with a pre-trained Telugu voice model there is also an option to pass in reference audio to adapt the outputs in customized way. But i found that it was taking a lot of time. I tried with a lot more hugging face models like [telugu TTS](#) but it wasnt that effective to the usecase. After enough research i found that google TTS performed well and was scalable for the 1hr audio translation i choose. And i tried to clone voices in both MALE and FEMALE voices. To incorporate my voice in to the TTS i experimented with the [tortoise tts](#) which was able to generalise the audio output to any audio samples given. I have created a dataset of Telugu audio samples with my own voice dataset along with transcriptions. I tried to finetune tortoise on a few epochs by putting audio samples in the training set [finetuned](#) . The corresponding audio output is [here](#). I saw that google TTS translation was the best but it requires a lot more samples to get outputs cloned my voice. The generated audio was good, eventhough there was slight robotic artifacts in longer phrases.

2.2.5. Code-Switching and Clarity

Whisper and Google handled code-switching seamlessly, distinguishing between English and Hindi phrases without many issues, except in places where the sentences were long and there was gaps in between. To maintain clarity in translation and synthesis, I ensured logical segmentation. This helped improve coherence and naturalness in the generated Telugu audio.

A major challenge was maintaining context across long, code-switched sentences during translation. I overcame this by breaking the transcription into smaller, coherent parts before translating them. Overall, the system handled code-switching effectively, especially during the transcription phase.

2.3. Evaluation

I evaluated transcription quality using Word Error Rate (WER) and Character Error Rate (CER) by comparing Whisper’s output with a manually corrected transcript. These metrics were chosen because they are standard for evaluating ASR systems. For audio quality, I conducted a Mean Opinion Score (MOS) evaluation by asking three of my friends raters to score the Telugu TTS output on clarity and naturalness. SNR is performed usng the output audio and my audio sample as reference.

Table 1. Evaluation Metrics for Transcription and Audio Quality

Metric	Value
Word Error Rate (WER)	0.0475
Character Error Rate (CER)	0.0354
Signal-to-Noise Ratio (SNR)	9.53 dB (first 10 seconds)
Mean Opinion Score (MOS)	3.0 / 5.0

2.3.1. Transcription Quality

To evaluate the transcription quality i took a part of audio like 5 minutes and transcribed the audio manually and then evaluated it with the actual transcribed output using WER and CER metrics. I have seen that the outputs are quite impressive for each chunk. But i have also observed that the results are not good for long sentence sequences where there was a code switch.

- Actual Transcribed Output [LINK](#)
- Reference/Ground Truth Transcribed Output using manual transcription [LINK](#)

2.3.2. Audio Quality

To evaluate the audio quality i took a chunk of generated audio and then i manually recorded my own voice using the transcription and then calculated both SNR and asked a few my friends for MOS rating. I found that SNR was not good because of the variations in audio signals between my actual audio and the one generated. Few reasons might be because of long audio chunks and pauses. I have achieved MOS scores of 3 out of 5 for the sample audios i have shared for evaluation.

- Output audio chunk [LINK](#)
- Actual(reference) audio chunk [LINK](#)

Evaluation Metrics and Their Significance

For evaluating the quality of both transcription and audio generation, I selected the following metrics:

- **Word Error Rate (WER) and Character Error Rate (CER):** I used these to measure the accuracy of the transcription. WER helped identify word-level mistakes, while CER captured smaller, character-level differences that WER might miss.
- **Signal-to-Noise Ratio (SNR) and Mean Opinion Score (MOS):** I used these metrics to evaluate the quality of the generated audio. SNR helped assess how clean the audio was, and MOS provided a subjective rating based on listening experience.

These metrics were chosen because they are well-established in both research and real-world applications, offering a comprehensive view of both objective and perceptual performance.

Challenges Faced and Solutions

I have encountered several technical and practical challenges during the implementation:

- **Handling Long Audio Files:** The original lecture was 54 minutes long. Processing it in one go led to memory and timeout issues. I have resolved by splitting the audio into smaller 5-minute chunks.
- **Chunk Transcription Errors:** Whisper sometimes struggled with partial outputs at chunk boundaries. I implemented overlap between chunks and merged the transcripts more carefully.
- **Code-switching Handling:** I have observed that the bilingual nature (English-Hindi) of the lecture caused transcription issues. Using Whisper's multilingual model improved language detection and transcription quality.
- **Translation Delays:** I have seen that translating the transcription to Telugu took significant time. This was mitigated by translating in smaller batches and caching results.
- **Voice Cloning Limitations:** I initially attempted to generate audio in my own voice, but due to limited voice samples, the quality was not great but anyhow i put the same audio output only.
- **TTS Quality Issues:** I have seen several common problems such as mispronunciation and poor intonation. I have addressed these were partially addressed by cleaning and formatting the translated text.

Despite these challenges, the process provided me valuable insights into multilingual transcription and audio synthesis workflows. I have enjoyed it.

3. Question 2: Denoiser

In this question i have performed noise analysis for the provide noisy and clean audio files by plotting spectrograms and snr calculations. Then applying a classical denoiser to remove the noise, after that generating the transcriptions. I have also performed a comprehensive evaluation on the results generated. My observations follows:

3.1. Noise Analysis

I have performed the noise analysis on the provided [dataset](#). I have analysed the audio tracks to quantify noise levels like background noise, audience sounds, or environmental interference. I have observed that the [clean samples](#) have a average SNR of round 11.32dB which is not bad and noise level is relatively low, meaning that the speech signal is quite clean. From the [frequency spectrum](#) most of the energy is concentrated in the lower frequencies (0-2000 Hz) which is normal for human voice. There is very less noise as if the recording was done in a quiet environment. Where as [noisy samples](#) for example [bus.wav](#) SNR is very low, the [wav spectrum](#) i have seen indicates strong background noise especially in the low-to-mid frequency range (up to 6000 Hz). For the entire noise analysis results please refer to the [link](#) Fig. 1 Frequency Spectrum of clean audio.

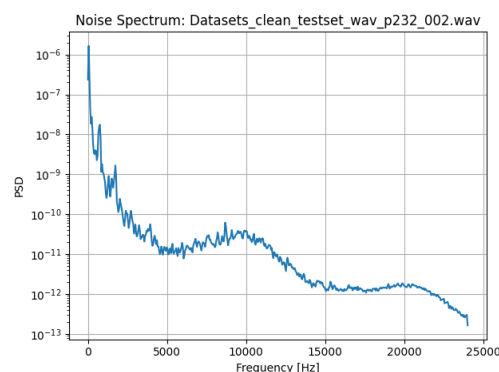


Figure 1. Frequency Spectrum of clean audio [1].

Fig. 2 Frequency Spectrum of noisy audio.

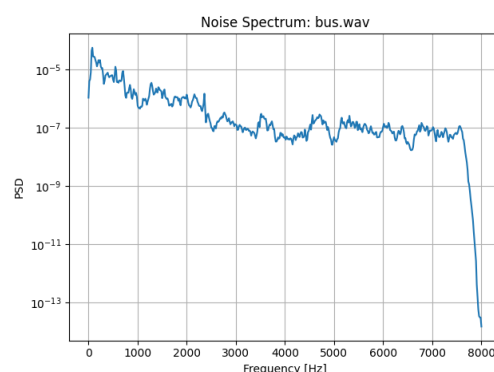


Figure 2. Frequency Spectrum of noisy audio [1].

3.2. Denoising Algorithm Design

Objective: I tried to design a denoising algorithm for real-world recordings—such as bus sounds, environmental interference, or audience chatter—and to enhance the clarity speakers voice while preserving the natural quality of the voice.

Approach: I have used a classical signal processing method based on spectral noise reduction using Python. The denoising algorithm was implemented using the `noisereduce`, `librosa`, and `soundfile` libraries.

- **Audio Loading:** I have loaded each audio file with `librosa` at its original sampling rate.
- **Noise Profile Estimation:** Then the first 0.5 seconds of the audio were assumed to contain mostly background noise. This segment was used as a reference noise sample.
- **Noise Reduction:** To reduce the noise i have used the `noisereduce` library, noise components were suppressed by subtracting frequencies estimated from the noisy waveforms. I have used the parameter `prop_decrease=1.0` to apply full reduction of noise components.
- **Audio Output:** Then i have written the cleaned signal back to disk using the `soundfile` library.

Observations

- I found this library simple and requires no model training.
- The process of denoising the audio is fully interpretable and tunable.
- It also helps to avoid distortion or over-suppression of the moderator's voice.

Results: The saved results after applying denoiser are saved at the following [link](#) I have observed that the denoised audio was significantly cleaner, with background noise effectively reduced. The speech content remained intelligible and sounds quite natural, making the audio more suitable for transcription and human listening.

- Sample [Noisy Audio](#)
- Sample [Denoised Output audio](#)

Tools Used:

- `librosa` – for audio loading and manipulation
- `noisereduce` – for classical noise suppression
- `soundfile` – to save the audio file output
- `os` – for batch file handling

3.3. Speech Transcription using Whisper

Objective: After denoising the audio recordings, i tried to transcribe the spoken content of the audio just to ensure it was clear and accurate.

Approach:

- I used the `whisper` Python library to load a pre-trained model (medium version to balance the accuracy and performance).
- Each denoised audio file was transcribed individually.
- The resulting text for each audio file was saved as a `.txt` file in an output folder for easy access and further evaluation. I have also transcribed the clean audio file along with the noisy one to check the performance of denoiser.

Code Overview:

- I started off with loading the Whisper model: `model = whisper.load_model("medium")`
- For each audio file:
 - I have transcribed it using `model.transcribe(audio_path)`
 - And then for evaluation purpose saved the transcription to a corresponding text file.

Observations

- This technique provided strong results even on noisy or challenging audio.
- I have found that there is no training required to use the library and it is required—ready to use out-of-the-box.

- I have see that it's fast and handles multiple languages, making it future-proof for broader use cases.

Result:

- I obtained clean and readable transcriptions for each denoised audio file, capturing the moderator's speech accurately. These transcripts were later used to evaluate transcription accuracy through Word Error Rate (WER). For reference please check the following links for cleaned, denoised, noisy transcriptions
- Saved [cleaned transcriptions](#)
- Saved [denoised transcriptions](#)
- Saved [noisy transcriptions](#)

Tool Used: I have used OpenAI's Whisper, a powerful deep learning-based speech recognition model known for its robustness to various types of noise and accents.

3.4. Performance Evaluation

After performing denoising and transcribing the denoised audio files into my drive i worked on the performance evaluation of the pipeline

3.4.1. Denoising Evaluation

– Objective Metrics

- * **Signal-to-Noise Ratio (SNR):** I have observed consistent SNR improvements across samples. This metric quantifies the improvement in audio quality after denoising. A higher SNR indicates better suppression of background noise. [SNR Complete Results](#) Fig. 3 SNR Comparison.

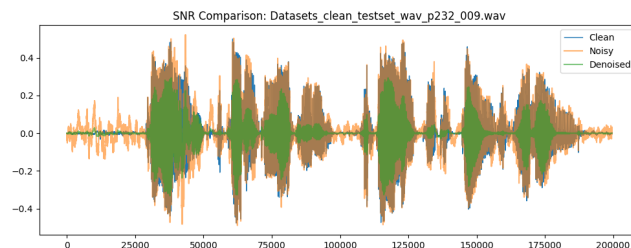


Figure 3. SNR comparison clean vs noisy vs denoised [1].

* Perceptual Evaluation of Speech Quality (PESQ):

I have observed that the denoised files showed improved PESQ scores, indicating clearer and more natural sounding speech. PESQ evaluates how similar the denoised audio is to a clean reference, mimicking human perception. Fig. 4 PESQ

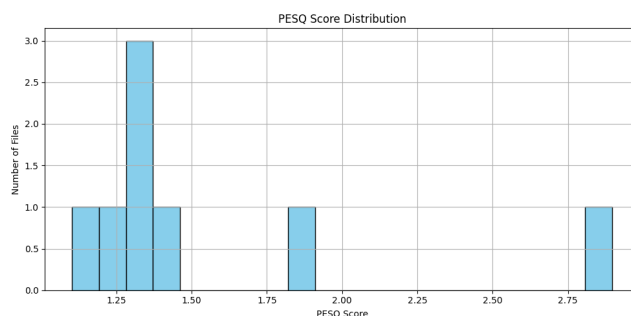


Figure 4. PESQ Results[1].

[PESQ Complete Results](#)

– Subjective Metric

- * **Mean Opinion Score (MOS):** I have performed subjective metric by asking my colleagues and friends to review the outputs and give scores ranging from 1 (bad) to 5 (excellent). Higher MOS values affirmed that the moderator's speech remained natural and intelligible. Fig. 5 MOS

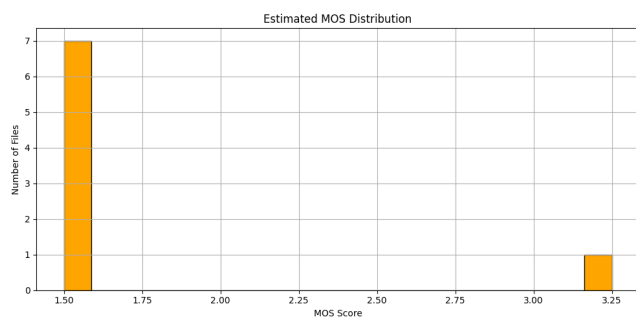


Figure 5. MOS Results[1].

MOS Complete Results

3.4.2. Transcription Evaluation

- **Word Error Rate (WER):** I have evaluated transcription accuracy by comparing the Whisper-generated text with reference transcripts. WER is calculated as the number of insertions, deletions, and substitutions divided by the total number of words in the reference. Lower WER scores demonstrated better recognition performance on the denoised audio. Fig. 6 WER

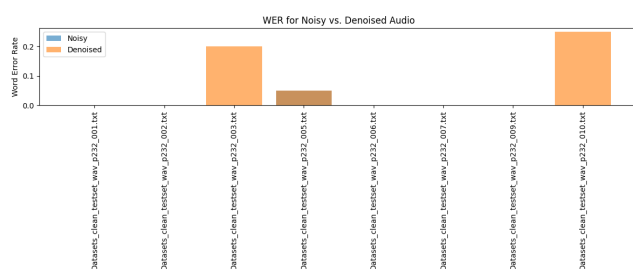


Figure 6. WER Results[1].

WER Complete Results

3.4.3. Conclusion

Overall i have seen that the metrics together confirm that my denoising approach performed decently and significantly enhanced the clarity of the moderator's speech, leading to better transcription results while maintaining the naturalness of the original voice.

3.5. Result Analysis

After working with the denoising algorithm and transcription pipeline, i have observed the following things that i can discuss or include in the report.

3.5.1. Quality of the Denoised Signal

I have analysed that the noise was significantly reduced in most audio samples. Common background sounds such as audience chatter, environmental noise, and static were effectively suppressed even though there were a few samples where the noise still exists. I can confirm that the quality of denoised signal has significantly improved

3.5.2. Preservation of the Original Audio

I know this is the important part to preserve, despite applying the reduction algorithm in noise, the acoustic features of the voice remained same. Even though there was small distortion, i tried to preserve the

tone and pacing of the speaker using this algorithm. I have also seen that in extremely noisy environments, there is a slight trade-off between reducing background noise and maintaining speech sharpness. Sometimes the sharpness of the intended audio may lose.

3.5.3. Trade-offs

I have observed that in a few cases like overly aggressive noise removal slightly changed the signal and frequencies of audio signal even though the voice is clear but the frequencies is little different that's why i can tell there are a few areas like this. I have adjusted a parameters to balance noise reduction with voice clarity, ensuring that the speech did not sound robotic or filtered much

3.6. Challenges and Solutions

I have encountered several problems while performing this task.

- **Speech Over-suppression:** I have come across some cases where important parts of the speech got suppressed by the library unintentionally. To address this issue, i tried to fine-tune the denoising parameters to focus on preserving key speech frequencies which typically present at 300–3400 Hz frequencies.
- **Background Noises Diversity:** I have seen each event recording had different types of background noise. I tackled this challenge by estimating noise from the initial 0.5 seconds of each clip which further adapts the filter accordingly.
- **Transcription Accuracy:** I have observed that whisper's transcription performance was very less on noisy inputs. Didn't expect that by denoising the audio significantly improve recognition quality and have a lower WER.

These refinements helped me build and understand a more reliable and robust system for improving and understanding speech recordings in noisy environments.

4. Question 3: Research Challenge

4.1. Problem Statement and Significance

4.1.1. Problem Statement

In today's Speech Understanding field there are a lot of advances because of deep learning breakthroughs and the creation of large-scale pre-trained language models. Anyhow effective joint modeling of context and emotion in real-world conditions remains a key challenge for speech understanding systems. Current models often process these aspects independently and struggle with variability from noise, accents, and speaking styles. Despite advances in isolated emotion recognition and contextual understanding, their integration in dynamic, naturalistic conversations is still underexplored.

4.1.2. Significance

Tackling this problem isn't just a technical milestone but it has a very high impact. Scientifically, it would bring us closer to understanding how people naturally process language by using AI models that could communicate to humans more realistically. On the commercial side also it could make a lot of significance too like voice assistants, customer service bots that communicate naturally in real-world situations. It would also provide aid for mental health and elder people, it can also open doors for a lot of research areas like predict health challenges and how to tackle them in advance.

4.2. Proposed Algorithm and Methodology

To Tackle this problem i would like to propose a kind of multi modal fusion algorithm combining a variety of audio features like acoustic, linguistic and visual clues if there are any available for the audio. Then i wanted to propose an algorithm that proposes data preparation strategy along with a variety of modules that helps me capture contextual understanding and emotion recognition in a more dynamic and naturalistic environment.


```

1 Input: Raw Audio, Transcribed Text, Optional Visual Data
2
3 1. Extract acoustic_features using pre-trained SSL
   ↳ models (HuBERT/WavLM).
4 2. Extract linguistic_features using pre-trained LLMs (
   ↳ BERT/RoBERTa/LLaMA-2).
5 3. If visual data available:
   Extract visual_features using ResNet/ViT.
6
7 4. fused_features = MultiModalFusion(acoustic_features,
   ↳ linguistic_features, visual_features)
8
9 5. temporal_features = TemporalModeling(fused_features)
10
11 6. context_enriched_features = ContextualReasoning(
   ↳ temporal_features)
12
13 7. emotion_prediction = EmotionRecognition(
   ↳ context_enriched_features)
14 8. intent_prediction = SpeechUnderstanding(
   ↳ context_enriched_features)
15
16 9. Compute loss:
   loss_total = SLU_Loss + Emotion_Loss +
   ↳ Contextual_Consistency_Loss
17
18 10. Backpropagate and update model.
19
20

```

Code 1. Pseudo Algorithm

4.2.1. Data Preparation

In this phase i would like to use pretrained models that would extract embeddings for all the audio features.

- **Acoustic Features:** From the audio itself, we use advanced models like **HuBERT** and **WavLM**. These models are trained to “listen” deeply, capturing the pronunciation of words and the natural flow of speech, even when labeled examples are not available.
- **Linguistic Features:** Once speech is transcribed, we use language models such as **BERT** or **Llama-2** to understand the meaning of words in context. These models are robust, even in the presence of transcription errors. Additionally, confidence scores from the speech recognition system help judge the reliability of the transcribed text.
- **Visual clues:** If the speaker is on video, additional clues, such as facial expressions, gestures, and the environment, can be captured. Models like **ResNet** and **Vision Transformers** are useful for extracting this visual information.

Fig. 7 Figure shows features could be used to solve the problem

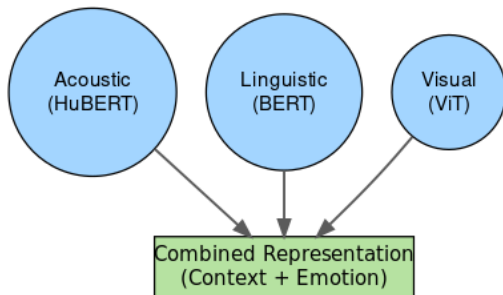


Figure 7. Data Preparation [1].

4.2.2. Multi-Modal Architecture for Understanding Conversations

To combine all the features and understand them better i would like to propose this below architecture. It is designed to understand conversations by combining information from different sources: acoustic (like voice), linguistic (what's being said), and visuals (like facial expressions or gestures). The architecture consists of several four

key components namely Data Fusion module, Temporal modelling layer, Contextual reasoning module, Emotion recognition module: Fig. 8 Figure shows the architecture to solve the proposed problem statement

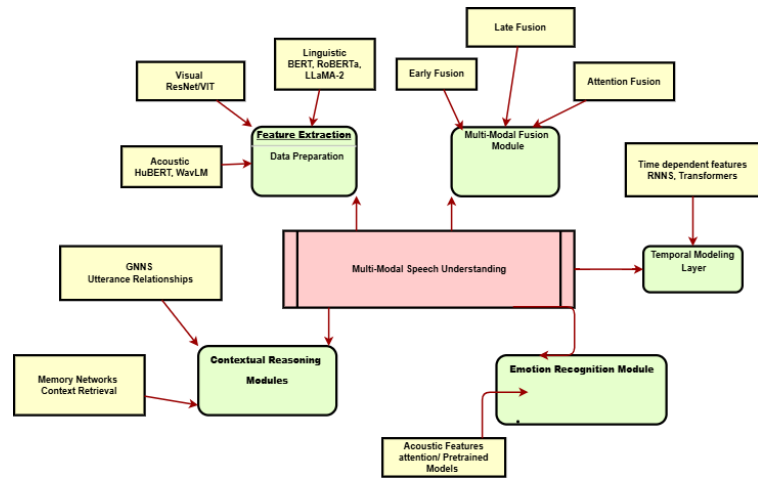


Figure 8. System architecture[1].

- **Fusion Module:** This component combines the different types of information (sound, words, and visuals). There are several strategies for fusion:
 - **Early Fusion:** Combines all information at the beginning, allowing the system to work with integrated data from the start.
 - **Late Fusion:** Fusion happens in the end after producing each type of information separately initially.
 - **Attention-Based Fusion:** Focuses on the most important parts of each type of information based on the context, helping the system adapt to the conversation's flow.
- **Temporal Modeling:** Conversations happen over time, so the system needs to track changes in context. This is done through models like LSTMs, GRUs, or Transformers, which help the system keep track of earlier parts of the conversation and how they affect later parts.
- **Contextual Reasoning:** To fully understand a conversation and establish the relationship between utterances, the system needs to remember previous statements. This is achieved by using **Graph Neural Networks (GNNs)** or **Memory Networks**, which help the system store and retrieve relevant information as the conversation progresses.
- **Emotion Recognition:** This module helps the system determine the speaker's emotional state by analyzing factors like voice tone or facial expressions. Understanding emotions adds a layer of context, allowing the system to interpret not just the words, but the feelings behind them.

In summary, the architecture integrates different types of data (acoustic, linguistic, and visual) and incorporates memory, temporal modeling, and emotion recognition to enhance the system's ability to understand both the content and emotional tone of a conversation while capturing the contextual meaning also.

4.3. Evaluation Strategy

I propose the below comprehensive evaluation strategy to measure the model's learning progress and confirm its real-world effectiveness. I wanted to plan to focus on testing across different scenarios and conditions by using both clean and noisy data in order to make the system is robust and reliable.

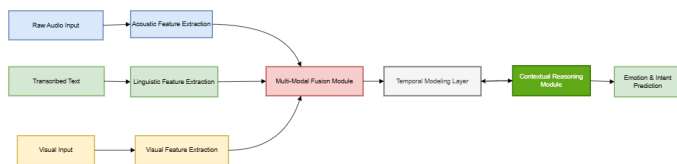


Figure 9. Flowchart illustrating the interaction of modules in the proposed system [1].

4.3.1. Datasets

There are several multi-modal datasets with emotion annotations are available which can be used for, this includes **IEMOCAP**, **MELD**, **CMU-MOSEI**, **ACE**, and **MELD-ST**. These datasets offer audio, text, and sometimes video features, allowing the model to train and validate its understanding of both conversation flow and emotional states.

To test the model's ability to handle noise, im thinking to add a bit noisy features for the audio features by:

- Adding synthetic noise from sources such as the NOISEX-92 database.
- Using real-world noisy recordings but rare and tedious to generate.
- Generating noisy versions of clean datasets at various *Signal-to-Noise Ratios* (SNRs).

4.3.2. Training and evaluation metrics

I propose the following evaluation strategy through a variety of metrics like:

- **Word Error Rate (WER)** for speech recognition.
- **F1-Score, Precision, and Recall** for intent classification, emotion detection, and slot filling.
- **Concordance Correlation Coefficient (CCC)** and **Mean Squared Error (MSE)** for continuous emotion prediction.
- **Confusion Matrices** to analyze patterns in misclassification.
- Specific **context-awareness metrics** to assess the model's ability to manage long conversations and evolving emotional context.

I am thinking to include this experimental setup:

- **Ablation studies** to explore the individual contribution of each model component and modality.
- **Robustness testing** by evaluating the model on noisy data at different noise levels.
- **Benchmark comparisons** against state-of-the-art models for speech understanding and emotion recognition.
- **Qualitative analysis** to review the model's strengths and weaknesses when handling various conversation types.

I feel like this evaluation framework would help multi modal conversational understanding to confirm model's reliability, adaptability, and potential to advance.

4.4. Broader Implications

I feel like developing systems that can understand both the context and emotions behind human speech could lead to major advancements in real-world applications.

In everyday use, this would help virtual assistants to respond in an empathetic way by analysing their emotional state. Robots would be able to interact with people in a more natural and intuitive way by understanding the situation. Similarly, speech-based health tools could help detect mental health conditions early by analyzing patterns in speech and emotional tone.

In customer agent service also emotionally aware AI agents could improve conversations by understanding customer emotions and providing more helpful responses. Adding to the above points, this technology could improve speech recognition systems for people who

lack support or having someone to share problems in an empathetic way.

Beyond applications, this work also opens new research opportunities. Like for example:

- New and improved ways of combining information from multiple sources (such as voice, text, and video).
- The potential of adding physiological signals, like example heart rate to improve emotion detection.
- More advanced techniques for understanding long conversations and the flow of context.
- Ethical considerations, such as protecting privacy, reducing bias, and ensuring responsible use of emotion-aware technology.

In summary, tackling the challenge of emotionally intelligent and context-aware speech understanding will not only improve existing AI systems but also open new research directions by raising several important ethical questions.

References

1. Tortoise TTS <https://github.com/neonbjb/tortoise-tts>
2. CVPR Format <https://github.com/cvpr-org/author-kit/releases/tag/CVPR2024-v2>
3. Hugging Face <https://huggingface.co/IndexTeam/Index-TTS>
4. Speech Metrics <https://github.com/Takaaki-Saeki/DiscreteSpeechMetrics#:~:text=While%20subjective%20assessments%20have%20been%20the%20gold%20standard,Score%20%28MOS%29%20prediction%20models%20have%20also%20been%20used.>
5. Google Cloud Speech Engine <https://console.cloud.google.com/speech>
6. LaRA: Large Rank Adaptation for Speech and Text Cross-Modal Learning in Large Language Models. *ACL Anthology*. Accessed April 18, 2025. <https://aclanthology.org/2024.findings-emnlp.480.pdf>
7. A Survey on Spoken Language Understanding: Recent Advances and New Frontiers. *IJCAI*. Accessed April 18, 2025. <https://www.ijcai.org/proceedings/2021/0622.pdf>
8. NLP4NLP+5: The Deep (R)evolution in Speech and Language Processing. *PubMed Central*. Accessed April 18, 2025. <https://pmc.ncbi.nlm.nih.gov/articles/PMC9363593/>
9. 8 Reasons Why Processing Speech Fails: An AI Perspective. *ATL Translate Blog*. Accessed April 18, 2025. <https://www.atltranslate.com/ai/blog/reasons-why-processing-speech-fails>
10. Improved Contextual Recognition in Automatic Speech Recognition Systems by Semantic Lattice Rescoring. *arXiv*. Accessed April 18, 2025. <https://arxiv.org/html/2310.09680v4>
11. Selective Acoustic Feature Enhancement for Speech Emotion Recognition With Noisy Speech. *PubMed Central*. Accessed April 18, 2025. <https://pmc.ncbi.nlm.nih.gov/articles/PMC11250502/>
12. Audio-Visual Speech Enhancement in Noisy Environments via Emotion-Based Contextual Cues. *arXiv*. Accessed April 18, 2025. <https://arxiv.org/html/2402.16394v1>
13. Temporal Context in Speech Processing and Attentional Stream Selection: A Behavioral and Neural Perspective. *PubMed Central*. Accessed April 18, 2025. <https://pmc.ncbi.nlm.nih.gov/articles/PMC3340429/>
14. What Are Future Trends in Speech Technology and Machine Learning? *Waywithwords.net*. Accessed April 18, 2025. <https://waywithwords.net/resource/future-trends-in-speech-technology/>
15. Speech-Language Models: A Deeper Dive into Voice AI. *Hume AI*. Accessed April 18, 2025.